From Scratch:
The Design and Implementation of a SAT Solver From Scratch

Oliver Lie

June 26, 2026

1 Abstract

This paper discusses the design and implementation methods of a SAT solver. It explores... INSERT METHODS HERE

2 Introduction

SAT solvers are amazing programs with one simple purpose: solve boolean algebra equations quickly.

2.1 CNF

CNF stands for "Conjunctive Normal Form," sometimes called "Clausal Normal Form." An equation in this form is a "conjunction of disjunctions," ie, it is an equation of "ands" ($\wedge$) in between clauses of "ors" ($\vee$). Each clause of "ors" contains one or more literals, or variables. Each literal has the option to be negated ($\neg$), however, no negation can appear in front of a clause itself. Some examples of CNF equations includes:

- $(A \vee B)$
- $(A \vee B) \wedge (\neg C)$
- $(A \vee B) \wedge (\neg C \vee D)$

Some examples that are not CNF:

- $\neg(A \vee B)$, where there is a NOT in front of a clause
- $\neg(A \vee B) \wedge (C)$, where is a NOT in front of a clause
- $(A \vee B \wedge C)$, where there is an AND within a clause
- $(A \wedge B) \vee (C)$, where it is in DNF form (read ahead)

2.2 DNF

DNF stands for "Disjunctive Normal Form." Just like in CNF form, a DNF equation has only three operators: conjunction ($\wedge$), disjunction ($\vee$), and negation ($\neg$). A DNF equation is in the form of a "disjunction of conjunctions," ie, it is an equation of "ors" ($\vee$) in between clauses of "ands" ($\wedge$). Each clause of "ands" contains one or more literals, each with the option of being negated ($\neg$), with no negation being in front of the clauses themselves. Some examples of DNF equations include:

- $(A \wedge B)$
- $(A \wedge B) \vee (C)$
- $(A \wedge B \wedge C) \vee (D \wedge E)$

Some examples that are not DNF:

- $(A \vee B)$, this is CNF form
- $\neg(A \wedge B) \vee (C)$, where there is a NOT in front of a clause
- $(A \wedge B \vee C) \vee (D \wedge E)$, where there is an OR within a clause

2.3 Defining our Input

For this SAT solver, we will use the DIMACS CNF format of input. DIMACS CNF format consists of seven rules:[1]

1. The file may begin with comment lines. The first character of each comment line must be a lower case letter "c". Comment lines typically occur in one section at the beginning of the file, but are allowed to appear throughout the file.
2. The comment lines are followed by the "problem" line. This begins with a lower case "p" followed by a space, followed by the problem type, which for CNF files is "cnf", followed by the number of variables followed by the number of clauses.
3. The remainder of the file contains lines defining the clauses, one by one.
4. A clause is defined by listing the index of each positive literal, and the negative index of each negative literal. Indices are 1-based, and for obvious reasons the index 0 is not allowed.
5. The definition of a clause may extend beyond a single line of text.
6. The definition of a clause is terminated by a final value of "0".
7. The file terminates after the last clause is defined.

There are some things about this syntax to be wary of:

1. The definition of the next clause normally begins on a new line, but may follow, on the same line, the "0" that marks the end of the previous clause.

[1] `https://people.sc.fsu.edu/~jburkardt/data/cnf/cnf.html`

2. In some examples of CNF files, the definition of the last clause is not terminated by a final '0'

3. In some examples of CNF files, the rule that the variables are numbered from 1 to N is not followed. The file might declare that there are 10 variables, for instance, but allow them to be numbered 2 through 11.

4. Comments can appear anywhere in the file, but the comment must be on its own line and it must start with 'c' [2]

For a quick example, here is a simple way to format a DIMACS CNF equation:

$$\begin{array}{l} \text{p cnf 4 2} \\ 1\ 2\ -3\ 0 \\ -1\ 4\ 0 \end{array}$$

And one with comments:

$$\begin{array}{l} \text{c this is a comment} \\ \text{p cnf 4 2} \\ \text{c this is another comment } 1\ 2\ -3\ 0 \\ -1\ 4\ 0 \end{array}$$

2.4 Input Processing

In order to process our input, we will provide a few APIs. We will firstly, disallow the header line. Ie, if no header is passed in, rather than throwing an error, we will simply infer the number of variables and clauses. We will also be very loose in terms of what input is considered valid. Our rules will be as follows:

1. Comments can appear anywhere in the file, but they MUST be on their own line and start with 'c'.

2. A header is not necessary, and if one is not provided, then the number of variables and clauses shall be inferred. However, if one is provided, then the number of variables and clauses MUST match the header.

3. All clauses, with the exception of the last clause, must end with '0', and can extend multiple lines.

4. All variables are numbered 1 - N, where N is the number of variables. Ie, if you say 10 variables, they must be labeled 1, 2, ..., N. And not 2, 3, ..., 11 or any other convention.

Thus, valid input can take on many forms:

$$\begin{array}{l} \text{p cnf 4 2} \\ 1\ 2\ -3\ 0 \\ -1\ 4\ 0 \end{array}$$

[2] https://jix.github.io/varisat/manual/0.2.0/formats/dimacs.html

4 2
1 2 −3 0
−1 4 0

p cnf 4 2
c this comment doesn't have to be at the top
1 2 −3 0
−1 4 0

p cnf 4 2
c this comment doesn't have to be at the top
1 2 −3 0
−1 4

c the first two numbers are still the number
c of variables and the number of clauses
p cnf 4 2 1 2 −3 0
−1 4 0

Now that we have fully defined our input rules, we can now define our API.

class SATSolver: public interface
SATSolver(string input);
int getNumVars();
int getNumClauses();
vector<vector<int>> getClauses();
optional<vector<bool>> getSolution();
void printSolution();
bool solveCNF();
class SATSolver: private interface
void parse(string equation);
int numVars;
int numClauses;
vector<vector<int>> clauses;
optional<vector<bool>> lits;

Our API takes in a string, either representing a file or a CNF equation and parses it into memory for our usage.
Pseudocode:

```
SATSolver(string input):
    lits ← no-value
    if (input is file) then
        string content ← readFile(input)
        parse(content)
    else
        parse(input)
void parse(equation):
    if (contains(equation, "p cnf")) then
        int[0...n] A
        for each line in input:
```

```
            if (startsWith(line, 'c')) then
                skip to next line
            else
                for each token in line:
                    A ← A::token
        numVars ← A[0]
        numClauses ← A[1]
        idx ← 2
        for i ← 0 to numClauses:
            int[0..n] clause
            while A[idx] ≠ 0
                clause ← A[idx]
                idx ← idx + 1
            clauses ← clause
else
        numVars ← 0
        numClauses ← 0
        int[0...n] A
        for each line in input:
            if (startsWith(line, 'c')) then
                skip to next line
            else
                for each token in line:
                    numVars ← max(token, numVars)
                    if token = 0 then
                        numClauses ← numClauses + 1
                    A ← A::token
        idx ← 0
        for i ← 0 to numClauses:
            int[0..n] clause
            while A[idx] ≠ 0
                clause ← A[idx]
                idx ← idx + 1
            clauses ← clause
```

While this algorithm isn't the most efficient, it is however readable enough to the point where anyone can understand what it's doing. SATSolver(...) checks if the input is a file, and if so, it reads in the file contents, then parses it as a CNF equation. Otherwise, it treats it as a CNF equation and then parses it as one. Then parse(...) extracts every numeric token that is not on a comment line, then it uses that numeric data to initialize its variables and data structures. It should be noted that this psuedocode does not handle bad inputs like a negative number of inputs and clauses, as it assumes perfect input. The actual implementation is up to you and me. To see how it was implemented in actual code, you are more than welcome to read the (commented since C++ isn't the most readable at times) source code.

3 Brute Force

The absolute simplest way of solving a SAT problem. In this section, we create our base SAT solver from scratch. We will not add any fancy features such as incremental solving, backtracking, or even basic contradiction detection... at first. In our simple brute force solver, we are given an input as defined above, and we simply solve by checking every possible combination of variable assignment. The pseudocode for this is actually just as simple as it sounds:

```
bool solveCNF():
    for each set of literal assignments:
        bool solved ← true
        for each clause in clauses:
            bool clause_true ← false
            for each literal_assignment in clause:
                clause_true ← clause_true or literal_assignment
            solved ← solved and clause_true
        if solved = true then
            lits ← current_set_of_literals
            return solved
    return false
```

Why this code works should be obvious. We generate up to 2^n sets of literals assignments (where n = numVars) then we check each one against our CNF equation. A clause is true if any one of the literals in the clause evaluates to true, and is false otherwise. And the whole CNF equation is true if every clause has at least one literal whose assignment within the clause is true. In that case, we store that solution, and return true, and in the case where none of the 2^n sets of literals assignments leads to a solvable outcome, we return false. A runtime analysis of this version of the algorithm shows that in the worst case (the equation is unsatisfiable and every combination must be checked) the algorithm runs in $O(2^n ms)$, where 2^n represents the number of combinations we need to check, m is the amount of time it takes to evaluate the whole equation, and s represents the amount of time it takes to generate one of the combinations that we check. This means in the worst case, the algorithm is abysmally slow. But what about the best case? Well, in the best case, our first literal assignment is the solution, so in that case it will run in $\Omega(ms)$. However, the likelihood of the best case ever happening is abysmally low, and the algorithm should be considered to always run in its worst case time.

Once again, you are welcome to read the source code for the brute force solver (with comments).

3.1 Brute Force Results

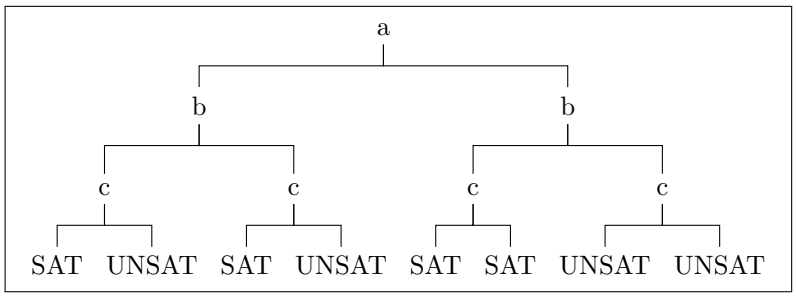
To see how well our current algorithm does, running some tests would be a good idea. Why run tests when we have already proven that our algorithm works "perfectly?" To see how quickly it runs of course. To test its speed (and as we add more features, to ensure correctness is kept), we use a database of problems from here: `https://www.cs.ubc.ca/~hoos/SATLIB/benchm.html`. We gave our current implementation 1000 SAT instances (uf20-91), each with

20 variables and 91 clauses each, all satisfiable. It took this baseline brute force algorithm 2,119,301 milliseconds, which is roughly 35.32 minutes. Normally in science you have to run an experiment at least three times for data accuracy. But you must understand that I don't want to give up an hour and a half of my life to an algorithm that I'm going to be improving upon anyways.

3.2 Recursive Backtracking Search

4 DPLL

The DPLL algorithm, or the Davis-Putnam-Logemann-Loveland algorithm, is a complete, backtracing-based search algorithm for deciding whether or not a CNF problem is satisfiable or not. How does it work? Imagine a binary tree of variables, each with two branches representing their assignment, one for $true$ and one for $false$. Suppose we don't know anything about our CNF equation. Ie, from looking at it, any literal could have any value, and we don't know how each assignment helps us in finding out whether or not the equation is satisfiable or not. How can we figure out if we can satisfy the equation or not without just brute forcing it? The answer is ~~making an ass out of you and me~~ to make assumptions.



The above picture of a DPLL decision tree (the outcomes may not map to a CNF problem, sue me).

Going back to our hypothetical, we don't know anything about anything. We could have a giant decision tree to explore with an exponential number of leaves to evaluate. How does making assumptions help us? Do we just run a random number generator and if it returns 1, we go "okay it's satisfiable" and false otherwise? No. We make assumptions about our literal assignments. Unlike brute forcing where we try every possible combination of variable assignments at once, in DPLL, we start with a literal, we'll call it A. There is nothing in the equation that indicates whether or not A should be true or false, either could be valid. So to tackle our problem, initially, we will assume A is true and see where that leads us. We will propagate that value throughout our clauses, and suppose there are two clauses where one forces $A = false$ and the other forces $A = true$. That's clearly a contradiction, so we've figured out that at the very least $A = true$ leads to a contradiction. So now, we've eliminated half of the search space immediately and we only need to consider the cases where $A = false$. This may not make total sense immediately, so instead of giving one explanation of the algorithm and building it, we will build up the algorithm and explain how each part works.

4.1 Recursive Backtracking

The first piece of this algorithm is a method that will recursively assign each literal's value as it traverses down the decision tree. This means we will need to change our private API to suit a new recursive method.

```
class SATSolver: public interface
  SATSolver(string input);
  int getNumVars();
  int getNumClauses();
  vector<vector<int>> getClauses();
  optional<vector<bool>> getSolution();
  void printSolution();
  bool solveCNF();
class SATSolver: private interface
  void parse(string equation);
  int numVars;
  int numClauses;
  vector<vector<int>> clauses;
  optional<vector<bool>> lits;
```

The old interface.

```
class SATSolver: public interface
  SATSolver(string input);
  int getNumVars();
  int getNumClauses();
  vector<vector<int>> getClauses();
  optional<vector<bool>> getSolution();
  void printSolution();
  bool solveCNF();
class SATSolver: private interface
  void parse(string equation);
  bool solve(vector<bool> assignment);
  int numVars;
  int numClauses;
  vector<vector<int>> clauses;
  optional<vector<bool>> lits;
```

The new interace, where *solve*() is a recursive method.
The pseudocode is as follows:

```
bool solveCNF():
    solve(lits)
bool solve(vector<bool> assignment[0...n]):
    if assignment.size = numVars then
        bool solved ← true
        for each clause in clauses:
            bool clause_true ← false
            for each literal_assignment in clause:
                clause_true ← clause_true or literal_assignment
```

```
            solved ← solved and clause_true
        return solved
    else
        assignment ← assignment::true
        if solve(assignment) then
            return true
        else
            assignment ← assignment[0...n-1]
            assignment ← assignment::false
            if solve(assignment) then
                return true
            else
                assignment ← assignment[0...n-1]
                return false
```

4.2 Recursive Backtracing - Results

Running this code on the same 1000 instances (uf20-91), it took 1846644ms, which is roughly 30.78 minutes. This is actually a pretty big improvement from our 35.32 minutes via brute forcing. Although this seems like a big gain in speed, it should be noted that at this point the algorithm is still just a fancier brute forcing algorithm.

4.3 Unit Clause Propagation

After recursive traversal and backgtracing, unit clause propagation is the next big piece of our "assume and explore" algorithm. Here's why it's useful: suppose you have a CNF equation like this: $(A) \wedge (A \vee B)$, or $(A) \wedge (\neg A \vee B)$. Notice how in order for the equation to be satisfied, A MUST be true by virtue of simply being by itself. Because of this, in the first equation, we see that B can either be true or false because just A being true is enough to satisfy everything. But in the second equation, $A = true$ FORCES $B = true$ as well, because $\neg A$ can't be true. On these small examples, this revalation doesn't seem important, but it's very important on larger examples, it can make a massive difference in speed.

But what if we don't have any unit clauses? Suppose we have the equation: $(A \vee B) \wedge (\neg A \vee C)$. There are no unit clauses, so we might think that we can't propagate anything. However, what if we assumed something was true? Visually speaking, since A appears twice in the equation across both clauses, A seems like a good pick to try this on. So let's assume $A = true$. What does that tell us? Well it immediately tells us that B can take on any value, but C must be true as well because $\neg A = false$. So now, in this slightly larger example, instead of brute forcing 8 cases (because 2^3 instances), we made 1 assumption and from there deduced one satisfiable assignment to the equation $(A = true, B = false, C = true)$.

Now that we've seen the power of assuming (don't do this in real life), let's modify our API and define pseudocode. We will add a new method for learning and assuming. To do this, we need to add in a new field as well.

```
class SATSolver: public interface
  SATSolver(string input);
  int getNumVars();
  int getNumClauses();
  vector<vector<int>> getClauses();
  optional<vector<bool>> getSolution();
  void printSolution();
  bool solveCNF();
class SATSolver: private interface
  void parse(string equation);
  bool solve(vector<bool> assignment);
  int numVars;
  int numClauses;
  vector<vector<int>> clauses;
  optional<vector<bool>> lits;
```

The old interface.

```
class SATSolver: public interface
  SATSolver(string input);
  int getNumVars();
  int getNumClauses();
  vector<vector<int>> getClauses();
  optional<vector<bool>> getSolution();
  void printSolution();
  bool solveCNF();
class SATSolver: private interface
  void parse(string equation);
  bool solve(vector<bool> assignment);
  vector<int> learn();
  int numVars;
  int numClauses;
  set<int> learned;
  vector<vector<int>> clauses;
  optional<vector<bool>> lits;
```

The new interface.

The idea of *learn*() is that we have a set of learned literal values and from that set of learned values, we want to deduce the values of other literals based off of what we know or assume. The easiest way to learn the value of a literal is to encounter a unit clause. A unit clause is a type of clause that contains only one literal, and because in CNF each must be satisfied, in order to satisfy the unit clause, the literal must have the value in the clause. So if we encounter 1, then literal 1 must be true. Likewise, if we encounter -1, -1 must be true, which means 1 must be false. After figuring out the value of 1, suppose we have the clause -12, and let's suppose $1 = true$. Because we learnt that $1 = true$, that means $-1 = false$, therefore, $2 = true$ must be true in order to satisfy that clause. Now suppose we the clause 12 and $1 = true$. What can we learn about 2? The answer is nothing! Because the clause is already satisfied by 1, nothing that we know forces 2 to be true or false, so we can't deduce anything.

So you can see how learning and propagation works. Before we modify the code of $solveCNF()$ and $solve()$, we will see what the pseudocode for $learn()$ is.

The pseudocode for $learn()$ is as follows:

```
vector<int> learn():
    set<int> learnt
    vector<bool> indices
    curSize ← learned.size()
    newSize ← curSize + 1
    while curSize ≠ newSize:
        curSize ← learned.size()
        for i ← 0 to clauses.size():
            if indices[i] ≠ true:
                at_least_one_true
                if clauses[i].size() = 1 and learned.contains(clauses[i][0]) = false then:
                    learnt.insert(clauses[i][0])
                    learned.insert(clauses[i][0])
                    indices[i] = true
                else
                    num_not_learnt ← 0
                    else_false ← false
                    index ← 0
                    for j ← 0 to clauses[i].size():
                        was_learned ← learned.contains(clauses[i][j]) or learned.contains(-clauses[i][j])
                        if was_learned = false and abs(clauses[i][j]) ¿ lits.size() then:
                            index = j
                            num_not_learnt ← num_not_learnt + 1
                            if num_not_learnt ¿ 1 then break
                        else:
                            forced_val
                            if was_learned and clauses[i][j] ¡ 0 then:
                                forced_val ← not learned.contains(clauses[i][j])
                            else if was_learned and clauses[i][j] ¿ 0 then:
                                forced_val ← learned.contains(clauses[i][j])
                            else forced_val ← lits[clauses[i][j]]
                            eval
                            if clauses[i][j] ¡ 0 then: eval ← not forced_val
                            else: eval ← forced_val
                            if eval = true then at_least_one_true = true
                        if num_not_learnt = 1 and not at_least_one_true then:
                            learnt.insert(clauses[i][j])
                            learned.insert(clauses[i][j])
                            indices[i] = true
        newSize ← learned.size()
    return vector from learnt
```

This pseudocode is a mouthful. But essentially, *indices* keeps track of which clauses $learn()$ knows are satisfied. Notice that it resets upon every call, so every time we want to learn something new, we need to check if it's already been learnt. If we encounter a unit clause, we immediately learn the literal's value, mark the clause as satisfied and continue. But in the non-unit clause case,

it's a bit harder to ensure that we can properly learn something new. The rules for a non-unit clause are: there is only one possible candidate to be learned, everything else has already been learned, and everything else within the clause evaluates to false. Ie, this literal's assignment MUST be true within the clause. So, all those conditionals in the else branch ensure beyond the shadow of a doubt that those conditions are met (I wrote the actual code when I was tired, and I'm making the pseudocode follow the actual code). (There is most definitely a more efficient encoding for what literals must be satisfied, and the learning algorithm could 100% be more efficiently written, but I'd rather have it work for now).

Now we can modify the pseudocode for $solveCNF()$ and $solve()$:

```
bool solveCNF():
    learn()
    solve(lits)
bool solve(vector<bool> assignment[0...n]):
    if assignment.size = numVars then
        bool solved ← true
        for each clause in clauses:
            bool clause_true ← false
            for each literal_assignment in clause:
                clause_true ← clause_true or literal_assignment
            solved ← solved and clause_true
        return solved
    else
        cur_lit = assignment.size() + 1
        if learned.contains(cur_lit) then:
            assignment ← assignment::true
            if solve(assignment) then:
                return true
            else:
                assignment ← assignment[0...n-1]
                return false
        else if learned.contains(-cur_lit) then:
            assignment ← assignment::false
            if solve(assignment) then:
                return true
            else:
                assignment ← assignment[0...n-1]
                return false
        else:
            learned.insert(cur_lit)
            assignment ← assignment::true
            learned_lits ← learn()
            if solve(assignment) then:
                return true
            else:
                assignment ← assignment[0...n-1]
                assignment ← assignment::false
                learned.remove(cur_lit)
```

```
for i ← 0 to learned_lits.size()
    learned.remove(learned_lits[i])
learned.insert(-cur_lit)
learned_lits ← learn()
if solve(assignment) then:
    return true
else:
    learned.remove(-cur_lit)
    assignment ← assignment[0...n-1]
    for i ← 0 to learned_lits.size()
        learned.remove(learned_lits[i])
    return false
```

For this section of the paper especially I would really recommend looking at the actual implementation for more comments. But essentially, we modified our solveCNF() method to first learn what it can. Then at the time of variable assignment, we check if we've already learned that literal's value, and if we have, we assign it its learnt value, then try to solve. If it fails, we remove that assignment (we will discuss this further) and return false. Otherwise, we assume that literal is true, try to learn from that, ie see what that literal being true would imply for the other literals, then try to solve. If solve() returns false after that, we undo everything we learned, and we now assume that literal is false instead and repeat the process. In the case where this literal being false also ends up returning false, we unlearn everything once again and we return false as well.

So why is it that when we have learnt a literal's assignment, when evaluating the equation, if the current literal's learnt value doesn't lead to a satisfiable assignment of variables, we undo its assignment? This is because there are two types of learned values: ones that are forced (explicit), and ones that are inferred (implicit). What are some kinds of explicit learned values? They are literals whose values are learned from unit clauses, or literals whose values are inferred directly from unit clauses like in our example. What are some kinds of implicit learned values? The ones from assumed/inferred literals. Those ones need to be undone. How do we know that if we learn something explicitly it won't be unlearned or undone? Because if something was explicitly learned, it's done in solveCNF(), and therefore can't be undone on a new call because solveCNF() doesn't pass on the literals it learned onto solve().

4.4 Unit Clause Propagation and Assumption - Results

After implementing unit clause propagation and assumptions into our DPLL algorithm, on the same 1000 instances of uf20-91, it took only 2363ms. This is a huge improvement. What used to take 30.78 minutes to solve now takes less than 2.5 seconds. As stated before, but may be forgotten, all 1000 problems in uf20-91 are satisfiable. But what about unsatisfiable problems? Before, we didn't test our SAT solver on any because our algorithm could only realize that a problem was unsatisfiable by testing every assignment. But now with unit clause propagation and assumption, we can figure out which branches could lead to a satisfiable assignment and which ones can't. Our second testing set is called UUF50.218.1000. It is a problem set of 1000 problems, each with 50

variables, and 218 clauses. Each problem is unsatisfiable. To solve just one using just recursive backtracking took over 10 minutes. However, with our new algorithm, to solve all 1000 cases, it took 646228ms, which is roughly 10.77 minutes. This again is a massive speedup, which makes testing our solver for both satisfiable and unsatisfiable cases viable, unlike before, which could only handle satisfiable cases.

4.5 Pure Literal Elimination

Suppose in our equation the literal -1 appears in multiple places. Perhaps it doesn't appear in a large amount of clauses, but nowhere in the CNF do we ever encounter the literal 1. This is a pure literal, and they have the special property of being able to be assigned the value that will make them satisfactory within their clauses immediately. While it may seem intuitive, or perhaps even "duh," how can we be sure of this? Suppose we have this equation:

$$(1 \vee 2 \vee \neg 3) \wedge (1 \vee \neg 2) \wedge (1 \vee 3)$$

We see that 1 is a pure literal, more specifically, it's a pure positive literal. Why can we immediately assign $1 = true$? The answer is really simple: it doesn't hurt. Saying $1 = true$ obviously only helps us, we can visually see that, but since $\neg 1$ doesn't show up anywhere, $1 = true$ doesn't make any other clauses less satisfiable, it only makes certain clauses more satisfied, and that's why it's okay.

An interesting thing about pure literals is that it can lead to other pure literals. Suppose we had the equation:

$$(1 \vee 2) \wedge (\neg 2 \vee 3) \wedge (3 \vee 4)$$

1, 3, and 4 are all pure literals, but let's only focus on 1 right now. Assigning $1 = true$ satisfies the first clause, and so we don't need to consider it anymore in future calculations. Assigning $1 = true$ effectively turns the equation into:

$$(\neg 2 \vee 3) \wedge (3 \vee 4)$$

because the assignment of 2 is now irrelevant with respect to 1. Notice how $\neg 2$ is a pure literal now. So we can say that $2 = false$ (to satisfy $\neg 2 = true$). So by assigning one pure literal, we were able propagate that too and figure out the rest of the equation without branching.